

CPSC 483: Introduction to Machine Learning

Instructor: Anli Ji

Fall 2026

Cal State Fullerton

Chapter 2

Data Preprocessing

- Exploratory data analysis
- Test-train split
- Sampling strategies
- Feature transforms
- Feature scaling
- Handling missing data
- Data processing pipelines

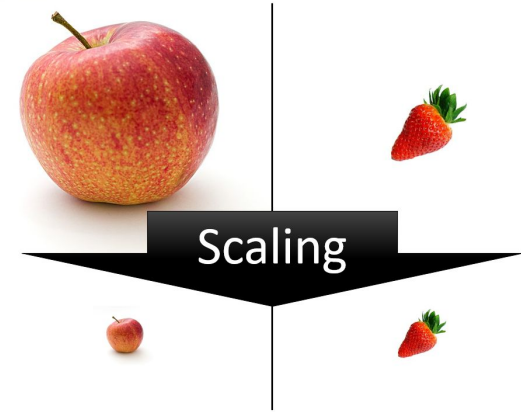
Feature Transformation - *Scaling*

Most ML algorithms don't perform well when input numerical attributes have very different scales

- will be biased toward ignoring the smaller unit features and focus more on the larger unit features

Example

- Age = [18, 80]
- Annual Income = [\$20,000, \$200,000]



Feature scaling is the process of transforming numerical features to a common scale without distorting differences in the range of values

Feature Transformation - *Scaling*

Min-Max Scaling (i.e., Normalization)

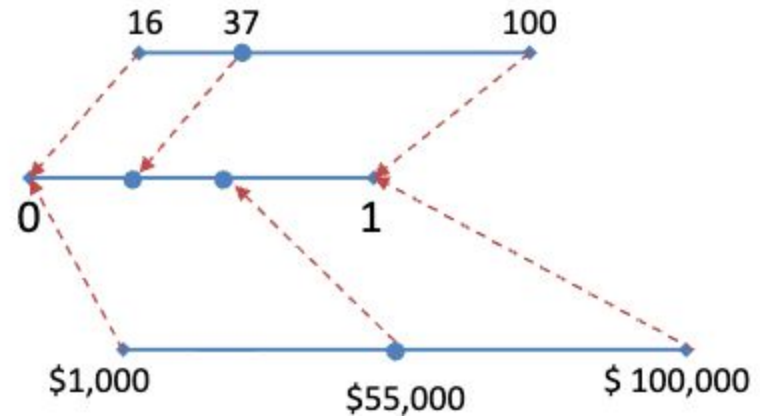
- Convert each feature independently into one comparable range (e.g., 0... 1)
- Percentile:
 - Proportion of the Min to Max range

Mathematically speaking:
$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Example:
$$\frac{37 - \min}{\max - \min} = \frac{37 - 16}{100 - 16} = 0.25$$

```
from sklearn.preprocessing import MinMaxScaler

min_max_scaler = MinMaxScaler(feature_range=(0, 1))
feature_scaled = min_max_scaler.fit_transform(original_feature)
```



Feature Transformation - *Scaling*

Min-Max Scaling (i.e., Normalization)

- What if different scale?
 - Maybe [-1, 1]?

Mathematically speaking:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \times (max_{range} - min_{range}) + min_{range}$$

Diagram annotations:

- Minimum values of features (points to x_{min})
- Maximum values of features (points to x_{max})
- Maximum target range (points to max_{range})
- Minimum target range (points to min_{range})

```
from sklearn.preprocessing import MinMaxScaler  
  
min_max_scaler = MinMaxScaler(feature_range=(-1,1))  
feature_scaled = min_max_scaler.fit_transform(original_feature)
```

Compresses normal values
But highly sensitive to outliers

Practice!!!

- What would be the output of applying Min-Max scaling to this dataset?
 - Range of $[0, 1]$
 - Range of $[-1, 1]$
- Do this by hand
- Do this with code
 - First create the *DataFrame*

Age	Income
16	10,000
28	2,000
30	6,000
23	5,000
60	25,000

Feature Transformation - *Scaling*

Standardization (i.e., Z-Score Scaling)

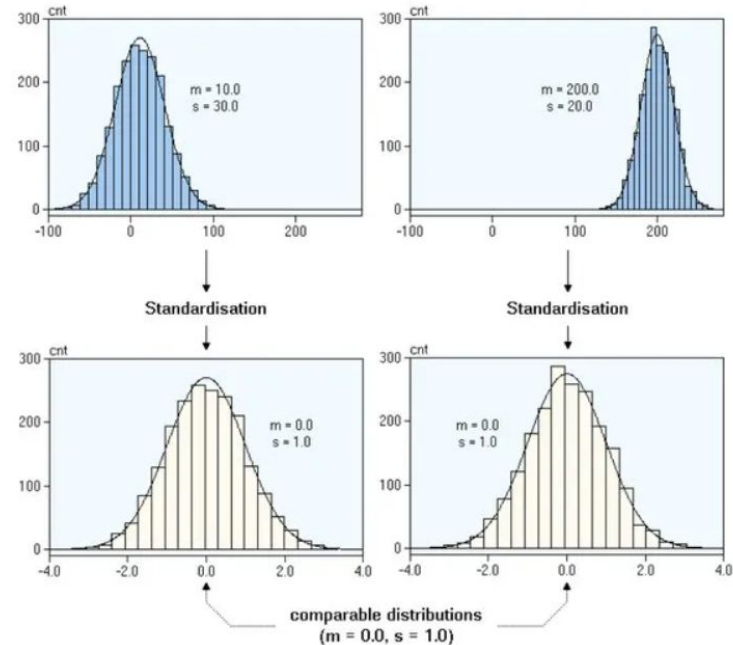
- Transforming data to have a mean of 0 and a standard deviation of 1
 - subtract the mean value
 - then divide by the standard deviation
- But range is not limited (can be very large positive or negative values)

Mathematically speaking:
$$z = \frac{x_i - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
```

```
std_scaler = StandardScaler()
```

```
features_scaled = std_scaler.fit_transform(original_features)
```



Feature Transformation - *Custom Transformers*

Other: Apply a numerical function to a feature (*Fancier implementation*)

- Combine features to get a new feature that is more related to the target variable
 - i.e., `log()`, `sqrt()`

```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```

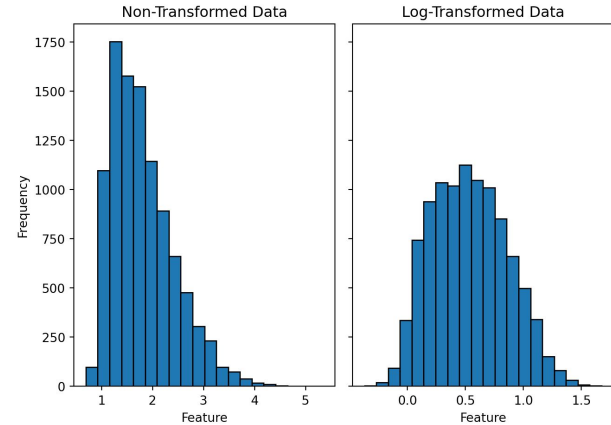
Example:

Ratio of two variables

- `housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]`
- `housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]`

Numerical function

- `housing["log_total_rooms"] = np.log( housing["total_rooms"] )`



Create a Test Set - *Sampling*

Split data into train-test subsets

- Do exploratory data analysis ONLY on the **train** subset
 - Avoid *data snooping bias*

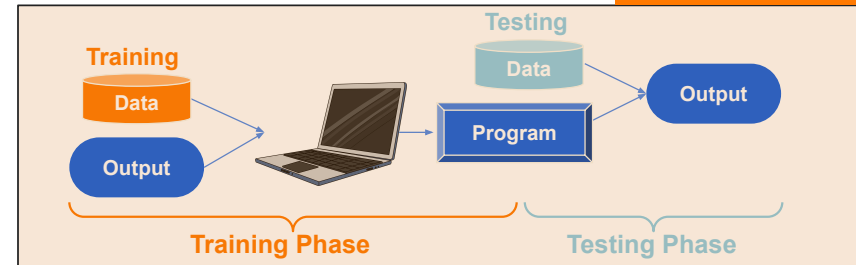
General rule:

- We want the train subset to be as large as possible
 - Better ML models with more data
- We want test subset to be large enough to be representative of the whole dataset
 - Different features appear in the same proportion

Typically,

- 80% for **training**, 20% for **testing**

Remember?



Create a Test Set - Sampling

What could go wrong?

- If we just pick first 80% for **training**, remaining 20% for **testing**

Feature/Feature Vectors				Label/Class
Petal Length	Petal Width	Sepal Length	Sepal Width	Species
6.2	2.9	4.3	1.3	Versicolor
6.3	3.3	6.0	2.5	Virginica
5.1	2.5	3.0	1.1	Versicolor
7.1	3.0	5.9	2.1	Virginica
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa

Training Data

Testing Data

Split for Evaluation

Something model NEVER seen before

Create a Test Set - *Sampling*

Random Sampling

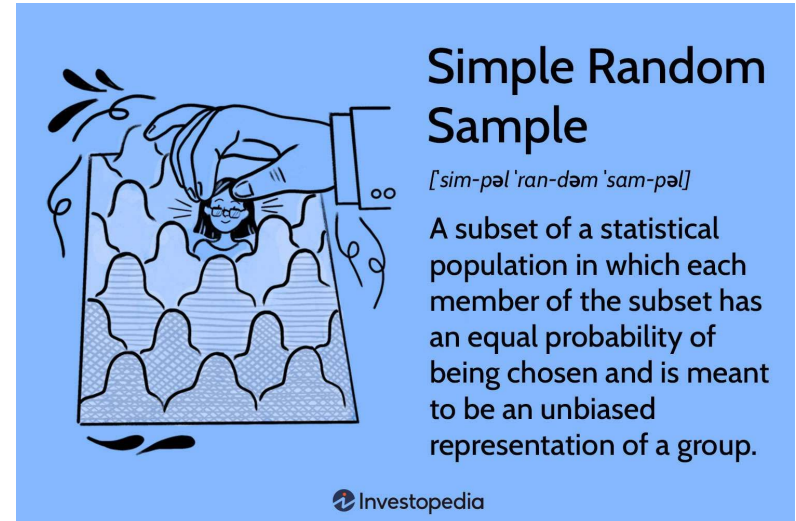
- A type of sampling method used to ensure every member of a population has an **equal chance** of being selected, reducing bias in data collection

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(myfulldata, test_size=0.2,
                                       random_state=42)
```

Problem

- Random sampling does NOT guarantee that test subset has features in the **same proportion** as the full data



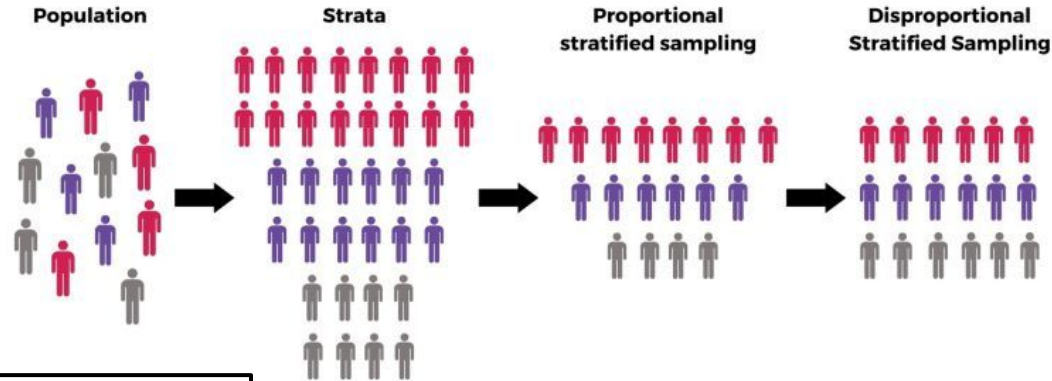
Reference:

<https://www.investopedia.com/terms/s/simple-random-sample.asp>

Create a Test Set - *Sampling*

Stratified Sampling

- Selecting a sample in which the population is first divided into homogeneous subgroups, (or *strata*) based on certain characteristics that are relevant to the study
- It take in an additional column parameter and ensures test and full dataset have the same proportion of the column values



```
train_set, test_set = train_test_split(myfulldata, test_size=0.2,  
                                       random_state=42, stratify=myfulldata["col_name"])
```

Reference:

<https://tgmresearch.com/stratified-sampling.html>

Create a Test Set - *Cross Validation*

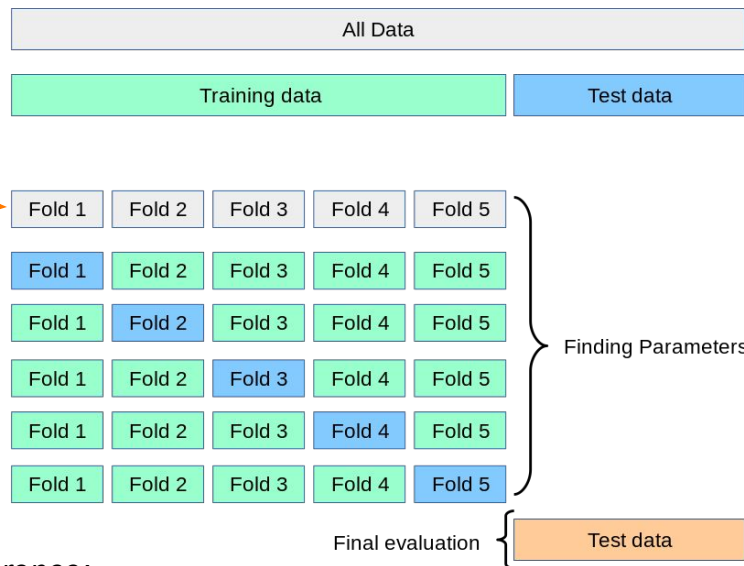
A single **train-test split** divides your data into one training set and one testing set

- while **cross-validation (CV)** divides your data into multiple parts (folds) to train and test the model iteratively

Common approach: ***k-fold cross-validation***

- k : the number of fold

```
from sklearn.model_selection import cross_val_score  
  
scores = cross_val_score(model, X_train, y_train, cv=5)
```



Reference:

https://scikit-learn.org/stable/modules/cross_validation.html